WormSizer-py: User Manual

A Python re-implementation of the measurement stage of **WormSizer** (Moore, Jordan & Baugh 2013, *PLoS ONE* 8(2): e57142). It takes **one binary mask per worm** and returns length, width, volume and surface area, plus quality-control (QC) numbers and diagnostic figures so you can judge each measurement.

This manual covers:

1. Quick start
2. How the calculation works
3. Parameter reference
4. Function arguments and command-line options
5. Output measures (every column of `results.csv`)
6. QC measures and flags
7. Reading the diagnostic figures
8. Practical guidance and limitations
9. Differences from the original WormSizer, and reference values

---

1. Quick start

One mask (a boolean 2D array):

```python
import imageio.v2 as imageio
from wormsizer_py import Params, analyze_mask, plot_worm, plot_debug

mask = imageio.imread("worm_mask.tif").astype(bool)
params = Params(um_per_px=2.0, interval_px=10, min_area_px=0)

results, skipped = analyze_mask(mask, params, name="worm1")
res = results[0]                          # one mask = one worm
print(res["row"]["length_um"], res["row"]["volume_pl"])

plot_worm(res, params, out_png="worm1_diagnostic.png", show=True)   # 6-panel overview
plot_debug(res, params, out_dir="worm1_debug", show=True)           # 5 step-by-step figu
```

A folder of TIFF masks:

python

```python
from wormsizer_py import Params, run_batch
df, failed = run_batch(r"C:\masks", Params(um_per_px=2.0, min_area_px=0),
                       out_dir="results", debug_n=3)
```

or from the command line:

```
python wormsizer_py.py C:\masks --out results --um-per-px 2 --debug-n 3
```

---

2. How the calculation works

Each mask goes through the same steps. The parameters that act at each step are in the right-hand column.

#	Step	What happens	Parameters involved
1	**Select the worm**	The mask is cropped around the worm and padded with background. Masks outside the area limits are skipped.	`min_area_px`, `max_area_px`, `pad_px`
2	**Clean the mask**	Optional closing, then enclosed holes are filled and only the largest connected piece is kept.	`close_radius`
3	**Skeleton**	The mask is thinned to a 1-pixel-wide skeleton. The **longest path** through it is kept (side branches are discarded = "pruning").	`min_skeleton_ratio`
4	**Smooth the path**	The pixel-staircase path is smoothed along its length.	`smooth_sigma_px`
5	**Extend the ends**	The path stops about one body radius short of each tip, so it is extended along its end direction until it leaves the mask.	`extend_tangent_px`, `max_extension_px`, `ray_step_px`
6	**Length**	Length = arc length of the extended path, converted with `um_per_px`.	`um_per_px`
7	**Width profile**	The path is sampled every `interval_px`. At each sample point, two rays are cast perpendicular to the path (one each side) until they leave the mask. Ray lengths = left and right radius. Their sum = width.	`interval_px`, `tangent_halfwin_px`, `ray_step_px`
8	**Volume and surface**	Between neighbouring samples the worm is treated as a **conical frustum** (a cone with its tip cut off). Volume and lateral area are summed over all frustums.	`um_per_px`

The central assumption (the only one) is that the worm's **cross-section is a circle**, so that its volume is the volume of the shape you get by spinning the width profile around the centerline.

Frustum formulas (r = radius, l = distance between samples, i = segment number):

```
frustum volume    = pi * l / 3 * (r_i^2 + r_i * r_(i+1) + r_(i+1)^2)
frustum area      = pi * (r_i + r_(i+1)) * sqrt(l^2 + (r_i - r_(i+1))^2)

VOLUME            = sum of frustum volumes     (reported in picoliters)
SURFACE AREA      = sum of frustum areas       (reported in um^2)
```

The last segment can be shorter than `interval_px` (it simply ends at the tip).

Units: 1 pl (picoliter) = 1000 μm³. A cube 10 μm on a side is 1 pl.

---

3. Parameter reference

All parameters live in the `Params` object (`Params(um_per_px=2.0, interval_px=5, ...)`). Only some are available as command-line flags; the rest are Python-only (see section 4).

Summary table

Parameter	Default	Unit	One-line meaning
`um_per_px`	2.0	µm / px	Image scale.
`interval_px`	10.0	px	Spacing of width samples along the worm (frustum size).
`min_area_px`	300.0 (CLI: 0)	px	Skip masks smaller than this.
`max_area_px`	∞	px	Skip masks larger than this.
`close_radius`	0	px	Morphological closing radius (0 = off).
`min_skeleton_ratio`	0.0	ratio	Skip masks whose skeleton is too branchy (0 = off).
`smooth_sigma_px`	3.0	px	Smoothing strength of the centerline.
`tangent_halfwin_px`	3.0	px	Window used to estimate the local direction (and so the ray direction).
`extend_tangent_px`	5.0	px	Length of path end used to decide the extension direction.
`max_extension_px`	40.0	px	Safety limit on how far the ends are extended.
`ray_step_px`	0.25	px	Step size when marching rays and extensions.
`pad_px`	12	px	Background padding around the cropped worm.

Details

`um_per_px`: image scale

Converts pixels to micrometers. **Length scales linearly with it, width linearly, and volume with its cube**: a 5% error in the scale gives about a 16% error in volume. Set it carefully (yours is 2.0). This is the only parameter that must be correct for absolute numbers to mean anything.

`interval_px`: sampling interval

How far apart the width samples (and therefore the frustums) are along the centerline.

- **Too small** (e.g. 1 px): every pixel-level wobble in the mask edge goes straight into the volume.
- **Too large** (e.g. half the worm): the frustums are too coarse to follow the tapering at the head and tail, so volume is underestimated.
- The paper found volume stable (CV about 2.3% across intervals) for **5 px up to roughly 10% of the worm length**, and used 10 px for its own data.
- **Rule of thumb for your data:** an interval of roughly 2–10% of the worm's length in pixels, never below about 5 px. At 2 μm/px a ~200 μm L1 larvae is only ~100 px long, so use 5; for ~1 mm adults (~500 px) 10 is fine.
- Check figure `05_sensitivity` (left panel) to see whether your choice matters.

`min_area_px` and `max_area_px`: area filter

Masks whose pixel area falls outside `[min_area_px, max_area_px]` are skipped and reported in the `skipped` list (the batch runner records them in `failed.csv`). The Python default of 300 px is the paper's GUI default for debris rejection. Because your masks are curated single worms, **set** `min_area_px=0` (the command line already defaults to 0).

`close_radius`: morphological closing

Applies a disk-shaped closing (dilate, then erode) of this radius before hole filling. It bridges gaps narrower than about twice the radius and smooths small notches. The paper's GUI used 5 for brightfield segmentations. For high-quality masks leave it at **0**; closing can also fill the narrow space between folded body parts and alter local width.

`min_skeleton_ratio`: branchiness filter

The paper's "minimum tubeness" filter. The ratio is *pixels on the kept longest path ÷ pixels in the full skeleton*. A smooth worm gives ≈ 1.0; side spurs from bumps, or a mask that is not tube-like, lower it. Masks below this value are skipped. Default 0 = no filtering; the ratio is still reported (`skeleton_ratio`) and flagged below 0.9.

`smooth_sigma_px`: centerline smoothing

The raw skeleton is a staircase of pixels, which is **longer than the true curve** (about 5–6% too long in my synthetic test). It is smoothed with a Gaussian of this width (in pixels along the path). The ends are handled so they do not shrink inward.

- `0`: raw pixel path (length is overestimated; angle plot is jagged).
- 2–5: usual range. Larger values give a straighter, shorter path.
- Too large: cuts the corners of tight bends and shifts the centerline off-center.
- See `02_centerline` (tangent angle panel) and `05_sensitivity` (middle panel).

`tangent_halfwin_px`: direction window

The local direction of the worm at a sample point is taken from the path position `half-window` before to `half-window` after it; the rays are cast perpendicular to that. Small values give noisy ray directions; large values average over bends, so rays tilt relative to the body and **overestimate width** in curved regions. It should be well below the bend radius. 2–6 px is sensible; check `05_sensitivity` (right panel).

`extend_tangent_px`: tip extension direction

The extension at each end follows the direction of the last this-many pixels of the (smoothed) path. Small values are noisy; large values are biased by curvature near the tip.

`max_extension_px`: extension limit

The extension continues until the path leaves the mask, but never more than this many pixels. The skeleton ends roughly one body-radius short of the tip, so the limit should exceed the worm's half-width at the tips. 40 px covers worms up to 80 px wide at the tip; raise it for very wide worms or high magnification.

`ray_step_px`: ray resolution

Step size when marching rays and extensions. The edge position is refined by interpolation, so accuracy is better than the step. Rarely needs changing.

`pad_px`: padding

Background rows/columns added around the cropped worm so that rays and extensions never run off the array. Rarely needs changing.

---

4. Function arguments and command-line options

`analyze_mask(mask_img, params, name="mask", mode="single")`

`mode` controls how the mask is split into worms:

`mode`	Behaviour
`"single"` (default)	The whole mask is **one worm**; stray fragments are dropped (largest connected piece kept).
`"binary"`	Connected components are separate worms.
`"labels"`	Each integer value is a separate worm (label image).
`"auto"`	`"labels"` if more than one non-zero value, otherwise `"binary"`.

It returns `(results, skipped)`. Each element of `results` holds the measured numbers in `res["row"]` plus the geometry used for plotting.

`run_batch(input_path, params, out_dir, recursive, plots, debug_n, show, iou_min)`

Argument	Default	Meaning
`input_path`	–	Folder of `.tif`/`.tiff` masks (case-insensitive) or one file.
`params`	`Params()`	Settings (section 3).
`out_dir`	`"wormsizer_out"`	Output folder.
`recursive`	`False`	Also search sub-folders.
`plots`	`True`	Save the 6-panel diagnostic figure for every mask.
`debug_n`	`0`	Save the 5 step-by-step figures for the first N masks (`-1` = all).
`show`	`False`	Open figures on screen (avoid for big batches).
`iou_min`	`0.9`	IoU below this raises the `low_iou` flag.

Output files: `results.csv`, `failed.csv` (only if something failed), `params.json`, `summary.png`, `plots/`, `debug/`.

Command-line flags

Flag	Equivalent	Default
`input`	`input_path`	required
`--out`	`out_dir`	`wormsizer_out`
`--um-per-px`	`um_per_px`	2.0
`--interval`	`interval_px`	10
`--min-area`	`min_area_px`	0
`--close-radius`	`close_radius`	0
`--smooth`	`smooth_sigma_px`	3.0
`--recursive`	`recursive`	off
`--no-plots`	`plots=False`	plots on
`--debug-n N`	`debug_n`	0
`--iou-min`	`iou_min`	0.9
`--show`	`show`	off

`max_area_px`, `min_skeleton_ratio`, `tangent_halfwin_px`, `extend_tangent_px`, `max_extension_px`, `ray_step_px` and `pad_px` are available only through `Params` in Python.

---

5. Output measures

Every row of `results.csv` is one mask. Columns appear in this order: `image`, `file`, the size measures, `model_iou`, `qc_flags`, then the remaining QC and bookkeeping columns.

Identification

Column	Meaning
`image`	File name without extension.
`file`	Full path of the mask file.
`centroid_x_px`, `centroid_y_px`	Center of mass of the worm mask, in pixel coordinates of the original image. Useful for locating the worm.

Size measures

Column	Unit	Meaning
`length_um`	µm	Arc length of the extended centerline from tip to tip.
`mid_width_um`	µm	Body **diameter** at the middle of the centerline (at half the length).
`mean_width_um`	µm	Average of the widths at all sample points (including the two near-zero tip samples, so it sits below `mid_width_um`; see note).
`volume_pl`	pl	**WormSizer volume**: sum of the conical frustums. The main result.
`cylinder_volume_pl`	pl	**Cylinder approximation**: pi × (`mid_width_um` / 2)² × `length_um` ÷ 1000.
`surface_area_um2`	µm²	Lateral surface area of the frustum model (end caps not included; the tips taper to a point anyway).
`mask_area_um2`	µm²	Area of the mask as seen in the image (pixel count × `um_per_px`²). This is the 2D silhouette area, **not** a body surface area.

Note on `mean_width_um`: because the first and last samples sit at the tips where width is about zero, the mean is pulled down by a few percent. If you want an average that does not depend on `interval_px`, `mask_area_um2 / length_um` is the silhouette-based mean width.

WormSizer volume vs cylinder-approximation volume

The two columns answer the same question (how big is this worm in 3D?) with different amounts of information.

- **Cylinder approximation** (the traditional method, e.g. Knight et al. 2002): measure one width at the middle of the worm and the length, and treat the worm as a straight can: `V =`

`pi * (w/2)^2 * L`. The middle is the **widest** part of the animal, so this assumes the whole body is as thick as it ever gets.

- **WormSizer volume** follows the width along the whole body, including the taper toward the head and tail.

Consequences:

- The cylinder volume is **systematically larger** than the WormSizer volume. In the original paper the cylinder estimate was about **1.38×** WormSizer's, at both L1 (66.3 vs 48.1 pl) and 48 h (2090 vs 1510 pl). In my tapered synthetic worms the ratio was about 1.4–1.45.
- The cylinder value also fluctuates more under imaging conditions, because it rests on a single width measurement (paper: repeat-measurement CV 19% for cylinder vs 11% for WormSizer at L1; 11% vs 8.9% at 48 h).
- **The ratio `cylinder_volume_pl / volume_pl` is a shape descriptor.** A ratio near 1 means a uniformly thick, tube-like worm. A larger ratio means more tapered. The summary figure titles the scatter plot with the mean ratio. If a worm's ratio is far from the rest of your population, look at its width profile.
- Use WormSizer volume for results. The cylinder value is provided for comparison with older methods and as a sanity check.

---

6. QC measures

These do not describe the worm; they describe **how trustworthy the measurement is**.

`model_iou`: radial-model overlap (IoU)

IoU means *Intersection over Union* (also called the Jaccard index). It scores how well two shapes overlap:

```
IoU = (area where both shapes are present) / (area where either shape is present)
```

- 1.0 = identical shapes; 0 = no overlap at all.
- Example: two shapes of 100 px each that share 90 px give IoU = 90 / (100 + 100 − 90) = 0.82.

Here: the code rebuilds a 2D silhouette from the measurements alone: it stamps a disc of the measured radius at every point along the centerline, and compares that rebuilt shape with your actual mask. The overlay is figure panel 4 (grey = both, red = mask only, blue = model only).

What it tells you:

- A **high IoU** means the centerline and width profile are a faithful description of your mask's outline, so the frustum volume is computed from a representative shape.
- A **low IoU** means the tube model does not fit the mask. Typical causes: a bad or off-center centerline, a mask that is not tube-like (kinks, coils, touching itself), a ray problem in a tight bend, or a mask with a stub or protrusion.

Rough guide (not a hard rule):

IoU	Reading
> 0.95	Excellent fit.
0.90 – 0.95	Good; typical for small worms.
< 0.90	Look at the figure (raises `low_iou`).

Small worms score a little lower even when everything is right, because a few pixels of edge quantization matter more on a 10-pixel-wide body. In my synthetic tests a perfectly modeled L1-sized worm scored about 0.92–0.94 while wide adults scored about 0.97.

What IoU does not tell you: it checks the 2D outline only. It cannot verify the assumption that the body cross-section is circular, nor that your mask boundary is in the right place (see section 8).

`skeleton_ratio`

Pixels on the kept longest skeleton path ÷ pixels in the whole skeleton. 1.00 = no side branches. Lower values mean spurs were pruned (rough or lumpy mask edge, or something attached to the worm). Flagged below 0.9.

`width_ray_vs_edt`

Two independent estimates of the body width at the interior sample points are compared:

- **Ray width** (used for the volume): distance to the edge on both sides, measured perpendicular to the centerline.
- **EDT width**: twice the distance from the centerline point to the *nearest* background pixel (the diameter of the largest circle that fits at that point), corrected by half a pixel.

The reported value is the **median ratio ray ÷ EDT** over the interior samples (the first and last two samples are excluded because tips are ill-defined).

- **≈ 1.0–1.1 is normal.** The ray width is slightly larger because the inscribed circle shrinks where the body tapers or curves.

- **> 1.3** suggests the centerline is off-center or the rays are tilted (try a smaller `tangent_halfwin_px` or `smooth_sigma_px`).
- **< 0.8** suggests rays are stopping early, for example at a notch or hole.
- Flagged outside 0.8–1.3.

`touches_border`

`True` if the mask touches the image edge. The worm is probably cut off, so length and volume are **underestimated**.

`had_holes`

`True` if the mask contained enclosed background regions (holes). They are filled before measuring. Small holes from segmentation are harmless, but a hole can also mean the worm is **coiled or crossing itself**, where the skeleton and width are unreliable.

`qc_flags`

A comma-separated list of the checks above that fired. Empty = nothing flagged.

Flag	Condition
`low_iou`	`model_iou` < `iou_min` (default 0.9)
`touches_border`	mask touches the image edge
`had_holes`	mask had enclosed holes
`branchy_skeleton`	`skeleton_ratio` < 0.9
`ray_edt_mismatch`	`width_ray_vs_edt` outside 0.8–1.3

The flags are heuristics, meant to tell you which masks to look at. A flagged mask can still be fine and an unflagged mask can still be wrong. Sorting `results.csv` by `model_iou` is a quick way to find the ones to inspect.

Numbers shown on figures only (not in the CSV)

- **interval CV** (6-panel figure): the coefficient of variation of the volume across a range of sampling intervals (2 px up to ~12% of the length). Low (a few percent) = the result does not depend on the interval you picked. The paper reports ~2.3%.
- **Ray/EDT width, tangent extension lengths, smoothing displacement** appear in the debug figures (section 7).

---

7. Reading the diagnostic figures

Six-panel figure (`plot_worm`, saved in `plots/`)

Panel	Shows	Check that
1	Full skeleton, pruned path, smoothed path, extension and sample points on the mask	The path runs down the middle and reaches both tips.
2	Contour, centerline and the sampled radii	Rays are perpendicular to the body and end on the edge.
3	Width profile (ray width vs 2×EDT width)	Smooth taper; the two curves agree.
4	Radial model vs mask (IoU)	Almost no red or blue.
5	Volume vs sampling interval	Curve is flat near your chosen interval.
6	The numbers and QC values	–

Debug figures (`plot_debug`, saved in `debug/<name>/`)

File	Shows	What to look for
`01_mask_skeleton`	Input mask; distance transform with skeleton; kept vs pruned branches	Skeleton on the bright ridge; few pruned pixels.
`02_centerline`	Zooms on both tips (tangent arrow, extension), final path colored by position, smoothing displacement, tangent angle along the path	Extension ends exactly at the tip; raw angle is jagged but the smoothed one is smooth.
`03_rays`	Rays colored by width, ray intensity profiles (edge = crossing 0.5), left vs right radius	Left and right radii similar; edges where the profile crosses 0.5.
`04_volume`	Worm straightened along its centerline with the frustum outline, volume per frustum, cumulative volume vs cylinder	Cyan frustum outline hugs the yellow measured edge.
`05_sensitivity`	Volume/length vs interval, smoothing σ and tangent window	Flat curves = robust result.

8. Practical guidance and limitations

Suggested starting settings

Situation	Suggested `interval_px`	Others
L1 larvae (~100 px long at 2 μm/px)	5	defaults
Adults (~500 px long at 2 μm/px)	10	defaults
Mixed stages in one run	5 (safe for all)	defaults

Keep the same settings for everything you intend to compare.

Suggested validation routine

1. Run the batch with `debug_n=3`.
2. Open `summary.png`, then sort `results.csv` by `model_iou` and open the plots for the lowest few.
3. Look at the `05_sensitivity` figure of a few typical worms: if volume moves by more than a few percent across reasonable settings, pick settings in the flat region and keep them fixed.
4. Check `cylinder_volume_pl / volume_pl` across the population: values should cluster (about 1.3–1.5 for a normal C. elegans, as in the paper).

Limitations to keep in mind

- **Circular cross-section.** Volume assumes the body is round. A worm lying on its side or a flattened animal violates this; IoU cannot detect it.
- **Mask boundary placement dominates the result.** Volume scales with width squared, so a small systematic boundary shift matters a lot on small worms. At 2 μm/px, an L1 is only ~10 px wide: if the mask is **1 px too generous on each side**, the width goes from 10 to 12 px and the volume rises by about **44%**. Segmentation tools differ in where they put the edge, so compare only masks produced by the same procedure, and treat absolute L1 volumes with caution.
- **Pixel quantization.** Small worms have few pixels across, so their width and volume are inherently noisier (the paper's repeat-measurement CV was higher at L1: 11% vs 8.9% at 48 h).
- **Self-touching or coiled worms** produce unreliable skeletons. They are flagged (`had_holes`) only when they enclose a hole.
- **Cut-off worms** (`touches_border`) are underestimated.
- **One bad scale ruins everything cubically.** Volume goes as `um_per_px` cubed.

9. Differences from the original WormSizer, and reference values

What is the same

Skeleton-based centerline, tip extension, perpendicular radii sampled at fixed intervals, summed conical frustums for volume (and, from WormSizer 1.2.0, for surface area). Units: μm and pl.

What is different

- The original ImageJ plugin also performs brightfield **preprocessing and thresholding** (rolling-ball background subtraction, "Minimum" threshold, closing). This code starts from your finished mask, so those steps are not included.
- It uses scikit-image's skeletonization rather than Fiji's Skeletonize3D.
- **Centerline smoothing is added** (`smooth_sigma_px`); it affects length mostly.
- "Mean width" here is the mean of the sampled widths; the original's exact definition may differ.
- The original GUI's manual pass/fail review is replaced by QC flags and diagnostic figures.

Because I could not read the plugin's Java source, treat small numeric differences from the original as expected. The method was validated against synthetic worms with known volume (errors of a few percent), not against the Java program itself.

Reference values from the paper (wild-type N2, Table 1)

Stage	Length	Mid width	WormSizer volume	Cylinder volume
L1 arrest (0 h)	~205 μm	~20 μm	~48 pl	~66 pl
Young adult (48 h)	~725 μm	~60 μm	~1510 pl	~2090 pl

Use them only as a plausibility check for your own stage; strain, temperature, food and imaging conditions all change these numbers.

Citation

Moore BT, Jordan JM, Baugh LR (2013) WormSizer: High-throughput Analysis of Nematode Size and Shape. *PLoS ONE* 8(2): e57142. doi:10.1371/journal.pone.0057142